

Ex No :	Hooks
Date :	

AIM:

To implement an application using useState, useEffect, useContext, useCallback, useMemo, useRef and Customized Hooks on react.

A) useState

1. You are developing a quiz app where the user's selected answer must be stored for each question. How would you use useState to track the chosen options?

PROGRAM:**App.jsx**

```
import React, { useState } from "react";
import "./App.css";
function App() {
  const questions = [
    {
      question: "Which language is mainly used with React?",
      options: ["Java", "Python", "JavaScript", "C++"]
    },
    {
      question: "Which Hook is used to manage state in React?",
      options: ["useEffect", "useState", "useRef", "useMemo"]
    }
  ];
  const [selectedAnswers, setSelectedAnswers] = useState({});
  const handleAnswer = (questionIndex, answer) => {
    setSelectedAnswers({
      ...selectedAnswers,
      [questionIndex]: answer
    });
  };
}
```

```
});  
};  
return (  
  <div className="page">  
    <div className="quiz-box">  
      <div className="header">  
        <div className="icon">🧐</div>  
        <div>  
          <h1>Quick Quiz</h1>  
        </div>  
      </div>  
      <div className="progress">  
        <span>  
          {Object.keys(selectedAnswers).length} / {questions.length}  
        </span>  
        <span>Questions Answered</span>  
      </div>  
      {questions.map((q, index) => (  
        <div className="question-card" key={index}>  
          <div className="question-number">  
            Question {index + 1}  
          </div>  
          <h2>{q.question}</h2>  
          <div className="options">  
            {q.options.map((option) => (  
              <button  
                key={option}  
                className={  
                  selectedAnswers[index] === option
```

```

      ? "option selected"
      : "option"
    }
    onClick={() => handleAnswer(index, option)}
  >
    <span className="radio">
      {selectedAnswers[index] === option ? "✓" : ""}
    </span>
    {option}
  </button>
  )})
</div>

{selectedAnswers[index] && (
  <div className="answer">
    ✓ Selected: {selectedAnswers[index]}
  </div>
)}
</div>
)})
<div className="footer">
  🎯 Choose one answer for each question
</div>
</div>
</div>
);
}
export default App;

```

OUTPUT:

Quick Quiz

0 / 2 Questions Answered

Question 1

Which language is mainly used with React?

☐ Java ☐ Python

☐ JavaScript ☐ C++

Question 2

Which Hook is used to manage state in React?

☐ useEffect ☐ useState

☐ useRef ☐ useMemo

Choose one answer for each question

Quick Quiz

2 / 2 Questions Answered

Question 1

Which language is mainly used with React?

☐ Java ☐ Python

☒ JavaScript ☐ C++

✓ Selected: JavaScript

Question 2

Which Hook is used to manage state in React?

☐ useEffect ☒ useState

☐ useRef ☐ useMemo

✓ Selected: useState

Choose one answer for each question

2. In a shopping app, you want to maintain the list of items added to the cart. How can useState help manage this dynamic array?

PROGRAM:**App.jsx**

```
import React, { useState } from "react";
```

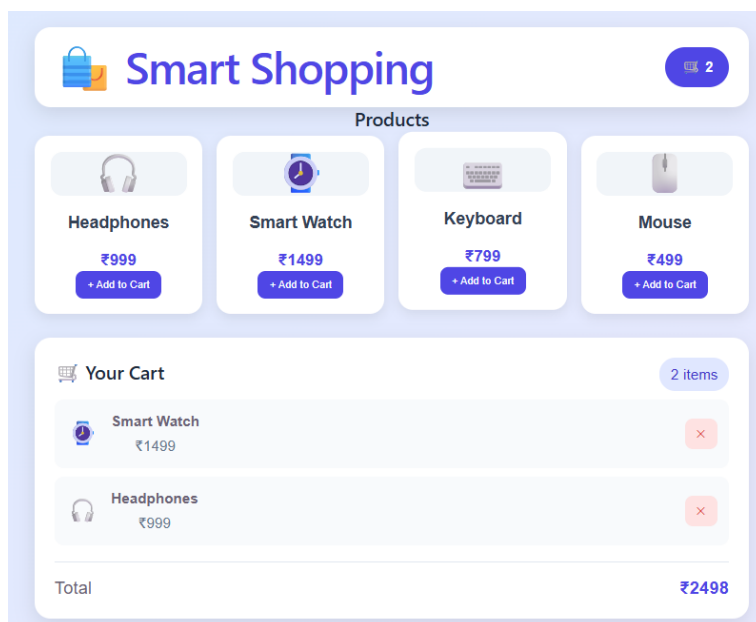
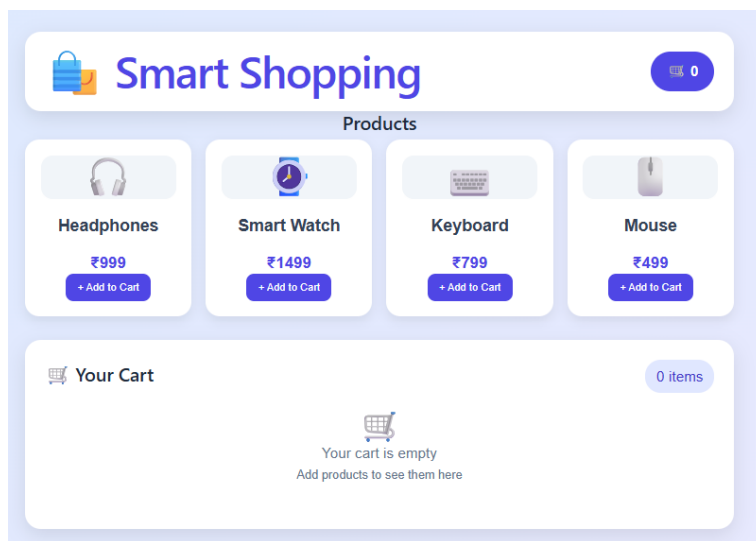
```
import "./App.css";
```

```
function App() {  
  const [cart, setCart] = useState([]);  
  const products = [  
    { id: 1, name: "Headphones", price: 999, icon: "🎧" },  
    { id: 2, name: "Smart Watch", price: 1499, icon: "🕒" },  
    { id: 3, name: "Keyboard", price: 799, icon: "⌨️" },  
    { id: 4, name: "Mouse", price: 499, icon: "🖱️" }  
  ];  
  const addToCart = (product) => {  
    setCart([...cart, product]);  
  };  
  const removeFromCart = (index) => {  
    const newCart = cart.filter((_, i) => i !== index);  
    setCart(newCart);  
  };  
  const total = cart.reduce((sum, item) => sum + item.price, 0);  
  return (  
    <div className="page">  
      <div className="shop">  
        <header>  
          <div>  
            <h1>🛒 Smart Shopping</h1>  
          </div>  
          <div className="cart-count">  
            🛒 {cart.length}  
          </div>  
        </header>  
        <h2>Products</h2>  
      </div>  
    </div>  
  );  
}
```

```
<div className="products">
  {products.map((product) => (
    <div className="product-card" key={product.id}>
      <div className="product-icon">
        {product.icon}
      </div>
      <h3>{product.name}</h3>
      <p>₹ {product.price}</p>
      <button onClick={() => addToCart(product)}>
        + Add to Cart
      </button>
    </div>
  ))}
</div>

<div className="cart">
  <div className="cart-title">
    <h2>&img alt="shopping cart icon" data-bbox="225 545 245 560"/> Your Cart</h2>
    <span>{cart.length} items</span>
  </div>
  {cart.length === 0 ? (
    <div className="empty">
      &img alt="shopping cart icon" data-bbox="195 690 215 705"/>
      <p>Your cart is empty</p>
      <small>Add products to see them here</small>
    </div>
  ) : (
    <div>
      {cart.map((item, index) => (
        <div className="cart-item" key={index}>
```

```
        <span className="item-icon">
          {item.icon}
        </span>
      <div>
        <b>{item.name}</b>
        <p>₹ {item.price}</p>
      </div>
      <button
        className="remove"
        onClick={() => removeFromCart(index)}
      >
        ✕
      </button>
    </div>
  )}
  <div className="total">
    <span>Total</span>
    <strong>₹ {total}</strong>
  </div>
</>
)}
</div>
</div>
</div>
);
}
export default App;
```

OUTPUT:

3. You are building a toggle button for showing/hiding a modal popup. How would you use useState to track the modal's visibility state?

PROGRAM:**App.jsx:**

```
import React, { useState } from "react";
import "../App.css";
function App() {
  const [showModal, setShowModal] = useState(false);
  return (
```

```
<div className="page">
  <div className="card">
    <div className="icon">  </div>
    <h1>Popup</h1>
    <p>
      Click the button below to open the modal.
    </p>
    <button
      className="open-btn"
      onClick={() => setShowModal(true)}
    >
      Open Modal
    </button>
  </div>
  {showModal && (
    <div className="overlay">
      <div className="modal">
        <div className="modal-icon">  </div>
        <h2>Welcome!</h2>
        <p>
          This popup is controlled using React's
          <b>useState</b> hook.
        </p>
        <button
          className="close-btn"
          onClick={() => setShowModal(false)}
        >
          Close
        </button>
      </div>
    </div>
  )}
```

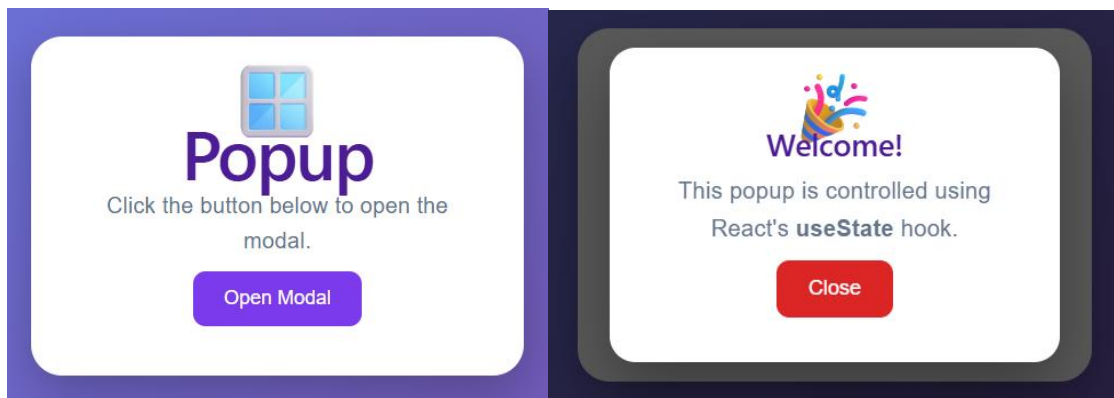
```

    </div>
  </div>
)}
</div>

);
}

export default App;

```

OUTPUT:**B) useEffect**

1. In a weather app, you need to fetch live weather data from an API whenever the page loads. How can you use useEffect to achieve this?

PROGRAM:**App.jsx:**

```

import React, { useState, useEffect } from "react";
import "./App.css";

function App() {
  const [weather, setWeather] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch("https://api.open-
meteo.com/v1/forecast?latitude=11.0168&longitude=76.9558&current=temperature_2m,win
d_speed_10m")
      .then((response) => response.json())

```

```

.then((data) => {
  setWeather(data.current);
  setLoading(false);
})
.catch((error) => {
  console.log(error);
  setLoading(false);
});
}, []);
return (
  <div className="page">
    <div className="weather-card">
      <div className="weather-icon">🌤️</div>
      <h1>Live Weather</h1>
      <p className="location">📍 Coimbatore, India</p>
      {loading ? (
        <div className="loading">
          ⌚ Loading weather...
        </div>
      ) : weather ? (
        <div className="weather-info">
          <div className="temperature">
            {weather.temperature_2m}°C
          </div>
          <div className="details">
            <div className="detail-box">
              🌀
              <span>Wind Speed</span>
              <b>{weather.wind_speed_10m} km/h</b>
            </div>
          </div>
        </div>
      ) : null}
    </div>
  </div>
)

```

```

    </div>

    <div className="detail-box">
      

      <span>Temperature</span>

      <b>{weather.temperature_2m}°C</b>
    </div>
  </div>

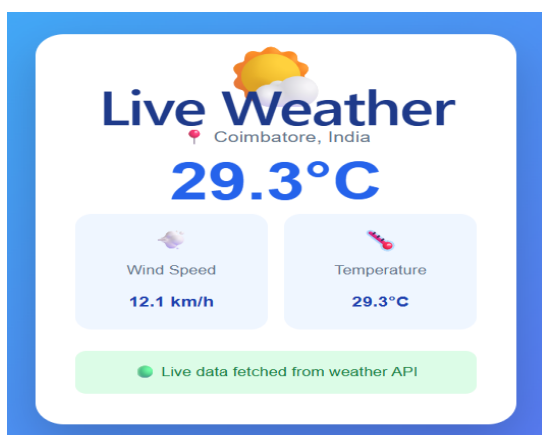
  <div className="status">
     Live data fetched from weather API
  </div>
</div>

):(
  <p className="error">
     Unable to fetch weather data
  </p>
  </div>
);
}

```

export default App;

OUTPUT:



2. You are tracking a user's scroll position to show a "Back to Top" button. How would you use useEffect to add and clean up the scroll event listener?

PROGRAM:

App.jsx:

```
import React, { useState, useEffect } from "react";
import "./App.css";
function App() {
  const [showButton, setShowButton] = useState(false);
  useEffect(() => {
    const handleScroll = () => {
      if (window.scrollY > 300) {
        setShowButton(true);
      } else {
        setShowButton(false);
      }
    };
    window.addEventListener("scroll", handleScroll);
    return () => {
      window.removeEventListener("scroll", handleScroll);
    };
  }, []);
  const backToTop = () => {
    window.scrollTo({
      top: 0,
      behavior: "smooth"
    });
  };
  return (
    <div className="page">
```

```
<header>

  <div className="logo">  ScrollSpace</div>

  <span>useEffect Demo</span>

</header>

<section className="hero">

  <div className="hero-icon">  </div>

  <h1>Scroll Down</h1>

  <p>

    Scroll the page to see the Back to Top button.

  </p>

</section>

<section className="content">

  <div className="card">

    <span>01</span>

    <h2>Track Scroll Position</h2>

    <p>

      React monitors the user's scroll position using

      a scroll event listener.

    </p>

  </div>

  <div className="card">

    <span>02</span>

    <h2>Show the Button</h2>

    <p>

      When the user scrolls more than 300 pixels,

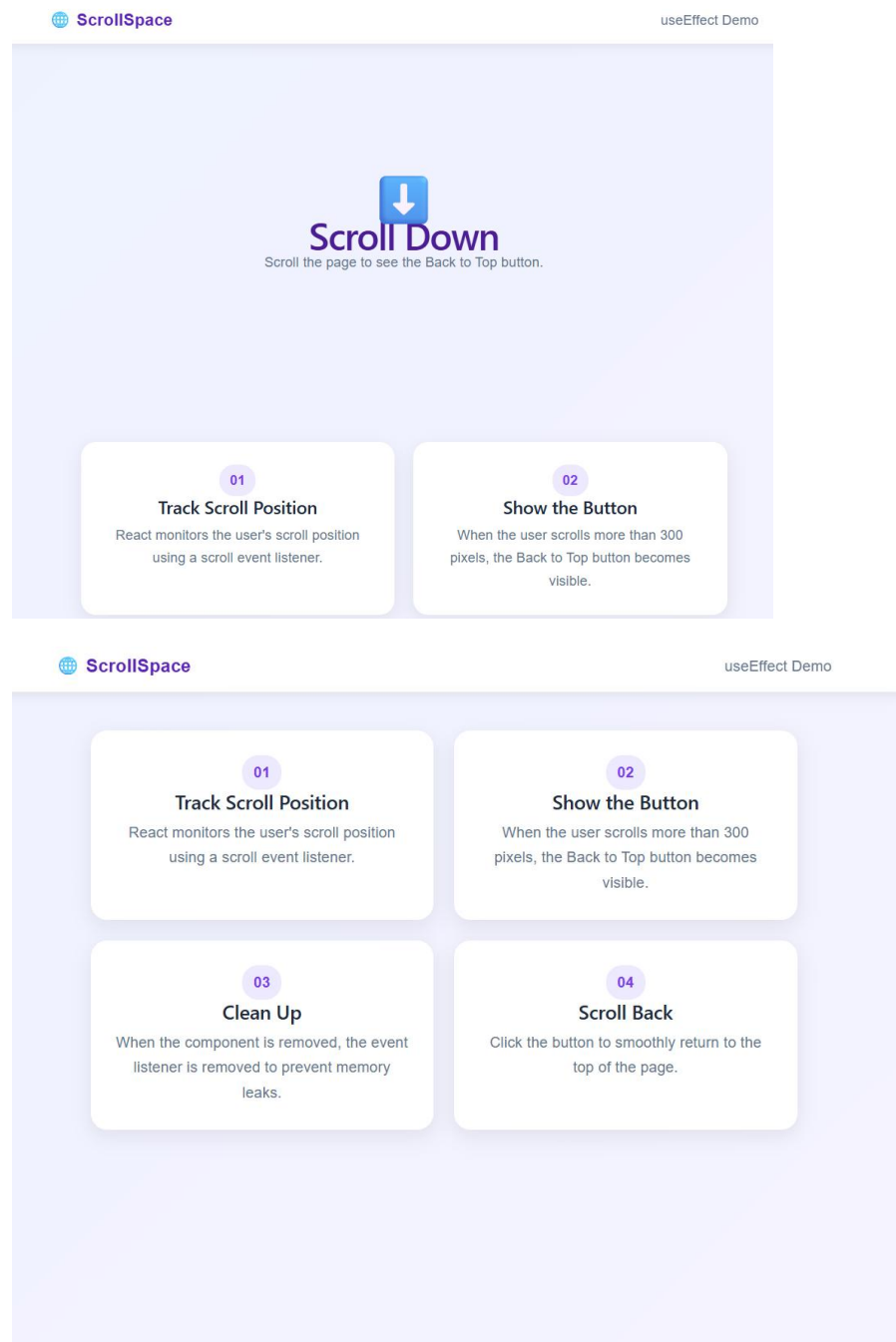
      the Back to Top button becomes visible.

    </p>

  </div>

  <div className="card">
```

```
<span>03</span>
<h2>Clean Up</h2>
<p>
  When the component is removed, the event listener
  is removed to prevent memory leaks.
</p>
</div>
<div className="card">
  <span>04</span>
  <h2>Scroll Back</h2>
  <p>
    Click the button to smoothly return to the top
    of the page.
  </p>
</div>
</section>
{showButton && (
  <button className="top-button" onClick={backToTop}>
    ↑
    <span>Back to Top</span>
  </button>
)}
</div>
);
}
export default App;
```

OUTPUT:

3. In a product details page, you need to fetch product info from the backend whenever the productId changes. How would you use useEffect to refetch data based on this dependency?

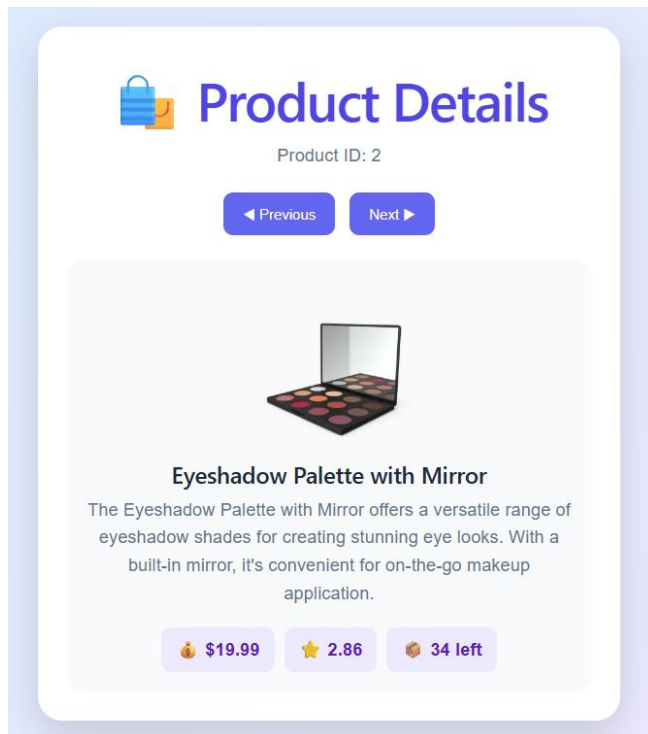
PROGRAM:**App.jsx:**

```
import React, { useState, useEffect } from "react";  
  
import "./App.css";
```

```
function App() {  
  const [productId, setProductId] = useState(1);  
  const [product, setProduct] = useState(null);  
  const [loading, setLoading] = useState(true);  
  useEffect(() => {  
    setLoading(true);  
    fetch(`https://dummyjson.com/products/${productId}`)  
      .then((response) => response.json())  
      .then((data) => {  
        setProduct(data);  
        setLoading(false);  
      })  
      .catch(() => {  
        setLoading(false);  
      });  
  }, [productId]);  
  return (  
    <div className="page">  
      <div className="product-card">  
        <h1>🛒 Product Details</h1>  
        <p className="subtitle">  
          Product ID: {productId}  
        </p>  
        <div className="buttons">  
          <button onClick={() => setProductId(productId - 1)}  
            disabled={productId === 1}>  
            ◀ Previous  
          </button>  
        </div>  
      </div>  
    </div>  
  );  
}
```

```
<button onClick={() => setProductId(productId + 1)}>
  Next ►
</button>
</div>
{loading ? (
  <div className="loading">
    ⌚ Loading product...
  </div>
) : product ? (
  <div className="details">
    <img
      src={product.thumbnail}
      alt={product.title}
    />
    <h2>{product.title}</h2>
    <p>{product.description}</p>
    <div className="info">
      <span>💰 ${product.price}</span>
      <span>★ {product.rating}</span>
      <span>📦 {product.stock} left</span>
    </div>
  </div>
) : (
  <p>❌ Product not found</p>
)}
</div>
</div>
);
```

```
}  
export default App;
```

OUTPUT:**C) useContext**

1. In a dashboard app, you need to share the logged-in user's profile information across multiple nested components. How can useContext help simplify this instead of prop drilling?

PROGRAM:**UserContext.jsx**

```
import React, { createContext } from "react";  
export const UserContext = createContext();  
export const UserProvider = ({ children }) => {  
  const user = {  
    name: "Jeen",  
    email: "jeenjangleena@gmail.com",  
    role: "Administrator",  
    location: "Chennai, India"  
  };  
};
```

```
return (  
  <UserContext.Provider value={user}>  
    {children}  
  </UserContext.Provider>  
);  
};
```

App.jsx:

```
import React, { useContext } from "react";  
import { UserContext } from "../UserContext";  
import "../App.css";  
function Header() {  
  const user = useContext(UserContext);  
  return (  
    <header>  
      <div className="logo">  
         My Dashboard  
      </div>  
      <div className="user-mini">  
        <div className="avatar">  
          {user.name.charAt(0)}  
        </div>  
        <div>  
          <b>{user.name}</b>  
          <small>{user.role}</small>  
        </div>  
      </div>  
    </header>  
  );  
}
```

```
function Profile() {  
  const user = useContext(UserContext);  
  return (  
    <div className="profile-card">  
      <div className="large-avatar">  
        {user.name.charAt(0)}  
      </div>  
      <h2>{user.name}</h2>  
      <p>{user.email}</p>  
      <span className="role">  
        {user.role}  
      </span>  
      <div className="profile-info">  
        <div>  
          <span> 📍 </span>  
          <b>Location</b>  
          <p>{user.location}</p>  
        </div>  
        <div>  
          <span> ✉️ </span>  
          <b>Email</b>  
          <p>{user.email}</p>  
        </div>  
      </div>  
    </div>  
  );  
}  
  
function Welcome() {  
  const user = useContext(UserContext);
```

```
return (  
  <div className="welcome">  
    <div>  
      <p className="label">WELCOME BACK 🙌</p>  
      <h1>  
        Hello, {user.name}!  
      </h1>  
      <p>  
        Here's what's happening with your dashboard today.  
      </p>  
    </div>  
    <div className="emoji">  
      🚀  
    </div>  
  </div>  
}  
function Dashboard() {  
  return (  
    <div className="dashboard">  
      <Header />  
      <main>  
        <Welcome />  
        <div className="content">  
          <Profile />  
          <div className="stats">  
            <div className="stat-card">  
              <span>📈</span>  
              <h3>Activity</h3>
```

```
<strong>85%</strong>
<p>Profile activity</p>
</div>
<div className="stat-card">
  <span>★</span>
  <h3>Status</h3>
  <strong>Active</strong>
  <p>Account status</p>
</div>
<div className="stat-card">
  <span>🔒</span>
  <h3>Role</h3>
  <strong>Admin</strong>
  <p>Access level</p>
</div>
<div className="stat-card">
  <span>🌐</span>
  <h3>Region</h3>
  <strong>India</strong>
  <p>User location</p>
</div>
</div>
</div>
</main>
</div>
);
}
function App() {
  return (
```

```

<Dashboard />

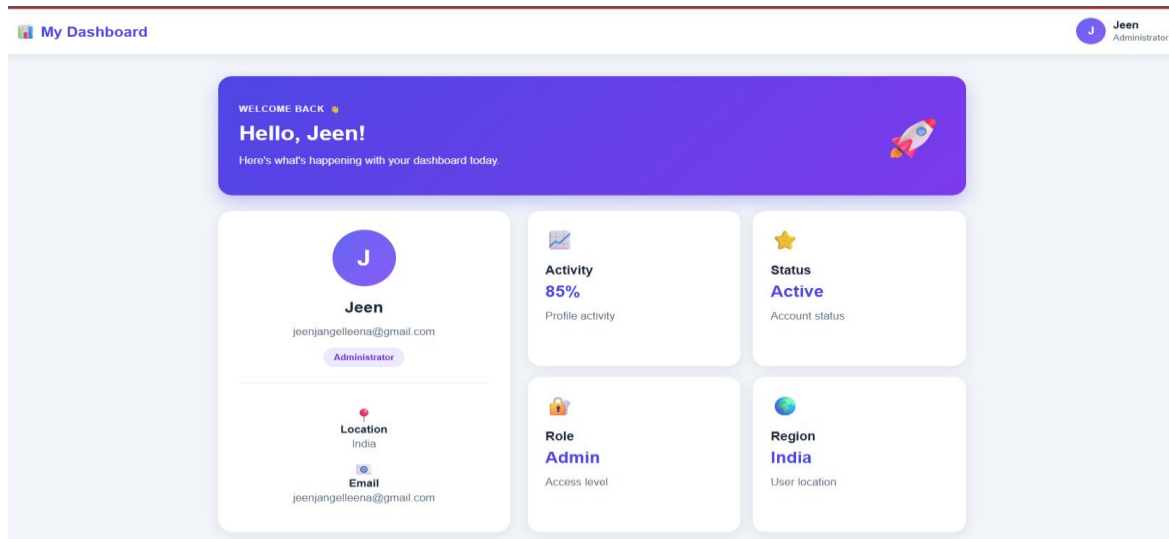
);

}

export default App;

```

OUTPUT:



2. A shopping cart has to display the total number of items in the header, sidebar, and footer simultaneously. How would you use useContext to manage this shared state?

PROGRAM:

App.jsx

```

import React, { useContext } from "react";
import { CartContext } from "../CartContext";
import "../App.css";

function Header() {
  const { cartCount } = useContext(CartContext);

  return (
    <header>
      <div className="logo">
        <img alt="ShopEasy logo" data-bbox="162 815 185 832"/> ShopEasy
      </div>
      <div className="header-cart">

```

```
     Cart
    <span>{cartCount}</span>
  </div>
</header>

);
}

function Sidebar() {
  const { cartCount } = useContext(CartContext);
  return (
    <aside>
      <h2>Menu</h2>
      <div className="menu-item">
         Home
      </div>
      <div className="menu-item">
         Products
      </div>
      <div className="menu-item cart-menu">
         My Cart
        <span>{cartCount}</span>
      </div>
    </aside>
  );
}

function Products() {
  const { addItem } = useContext(CartContext);
  return (
    <main>
      <div className="welcome">
```

```
<h1>Welcome to ShopEasy 🙌</h1>
<p>Add products to your shopping cart.</p>
</div>
<div className="products">
  <div className="product">
    <div className="product-icon"> 🎧 </div>
    <h3>Headphones</h3>
    <p>₹999</p>
    <button onClick={addItem}>
      Add to Cart
    </button>
  </div>
  <div className="product">
    <div className="product-icon"> 🕒 </div>
    <h3>Smart Watch</h3>
    <p>₹1,499</p>
    <button onClick={addItem}>
      Add to Cart
    </button>
  </div>
  <div className="product">
    <div className="product-icon"> 🎒 </div>
    <h3>Backpack</h3>
    <p>₹799</p>
    <button onClick={addItem}>
      Add to Cart
    </button>
  </div>
</div>
```

```
    </main>

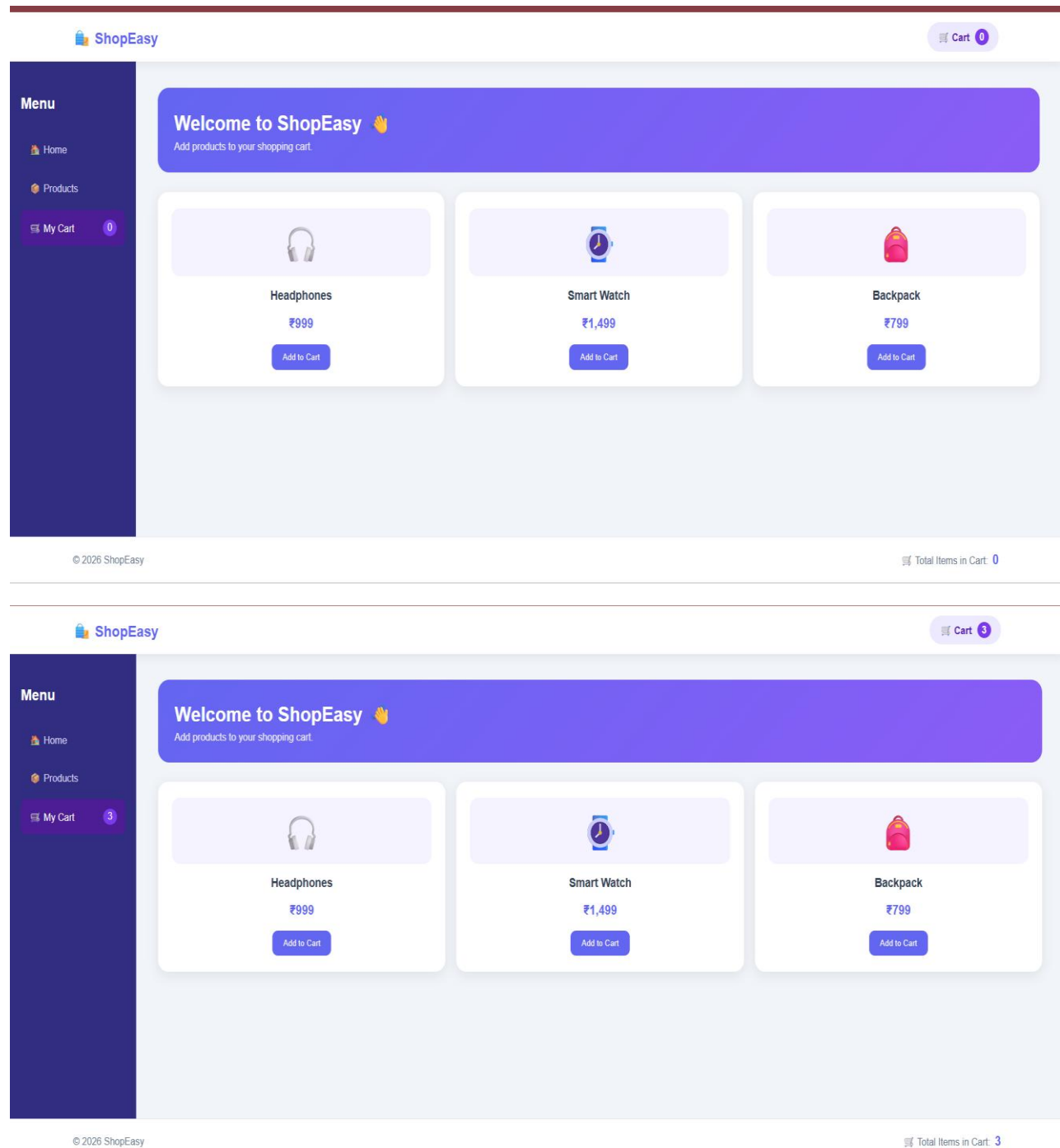
  );
}

function Footer() {
  const { cartCount } = useContext(CartContext);
  return (
    <footer>
      <div>
        © 2026 ShopEasy
      </div>
      <div>
        🛒 Total Items in Cart:
        <strong>{cartCount}</strong>
      </div>
    </footer>
  );
}

function App() {
  return (
    <div className="app">
      <Header />
      <div className="layout">
        <Sidebar />
        <Products />
      </div>
      <Footer />
    </div>
  );
}
```

export default App;

OUTPUT:



D) useCallback

1. You have a list of users, and each user has a “Follow” button. The click handler is defined inside the parent component, causing performance issues. How would you optimize it using useCallback?

PROGRAM:

App.jsx:

```
import React, { useState, useCallback, memo } from "react";
import "./App.css";

const UserCard = memo(function UserCard({ user, onFollow }) {
  console.log("Rendered:", user.name);
  return (
    <div className="user-card">
      <div className="avatar">
        {user.name.charAt(0)}
      </div>
      <div className="user-info">
        <h3>{user.name}</h3>
        <p>{user.username}</p>
        <span>{user.followers}</span> followers</div>
      <button
        className={user.following ? "following" : ""}
        onClick={() => onFollow(user.id)}
      >
        {user.following ? "✓ Following" : "+ Follow"}
      </button>
    </div>
  );
});

function App() {
  const [users, setUsers] = useState([
    { id: 1,
      name: "Ananya",
      username: "@ananya",
```

```
    followers: 1250,
    following: false
  }, { id: 2,
    name: "Rahul",
    username: "@rahul",
    followers: 980,
    following: false
  }, { id: 3,
    name: "Priya",
    username: "@priya",
    followers: 2340,
    following: false
  }, {
    id: 4,
    name: "Arjun",
    username: "@arjun",
    followers: 1750,
    following: false
  }
]);

const handleFollow = useCallback((userId) => {
  setUsers((previousUsers) =>
    previousUsers.map((user) =>
      user.id === userId
        ? {
            ...user,
            following: !user.following,
            followers: user.following
              ? user.followers - 1
```

```
      : user.followers + 1
    }
    : user
  )
);
}, []);
return (
  <div className="page">
    <div className="container">
      <header>
        <div>
          <h1> 👥 ConnectHub</h1>
        </div>
        <div className="people-count">
          👤 {users.length} Users
        </div>
      </header>
      <div className="title">
        <h2>People You May Know</h2>
        <p>Follow users to stay connected with them.</p>
      </div>
      <div className="user-list">
        {users.map((user) => (
          <UserCard
            key={user.id}
            user={user}
            onFollow={handleFollow}
          />
        ))}
      </div>
    </div>
  </div>
);
```

```
</div>
```

```
<div className="info-box">
```

```
  ⚡ <b>Optimized with useCallback</b>
```

```
  <br />
```

The Follow handler keeps the same function reference
between renders.

```
</div>
```

```
</div>
```

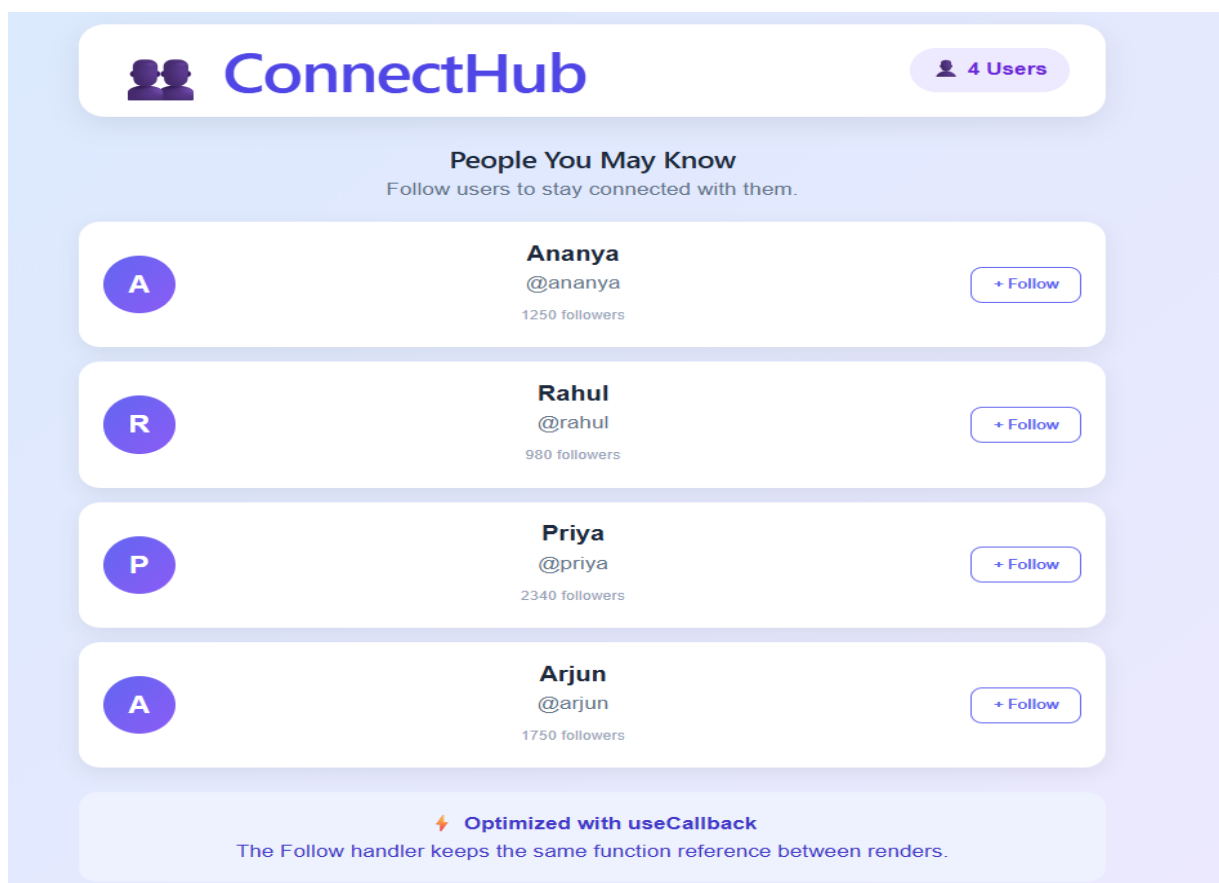
```
</div>
```

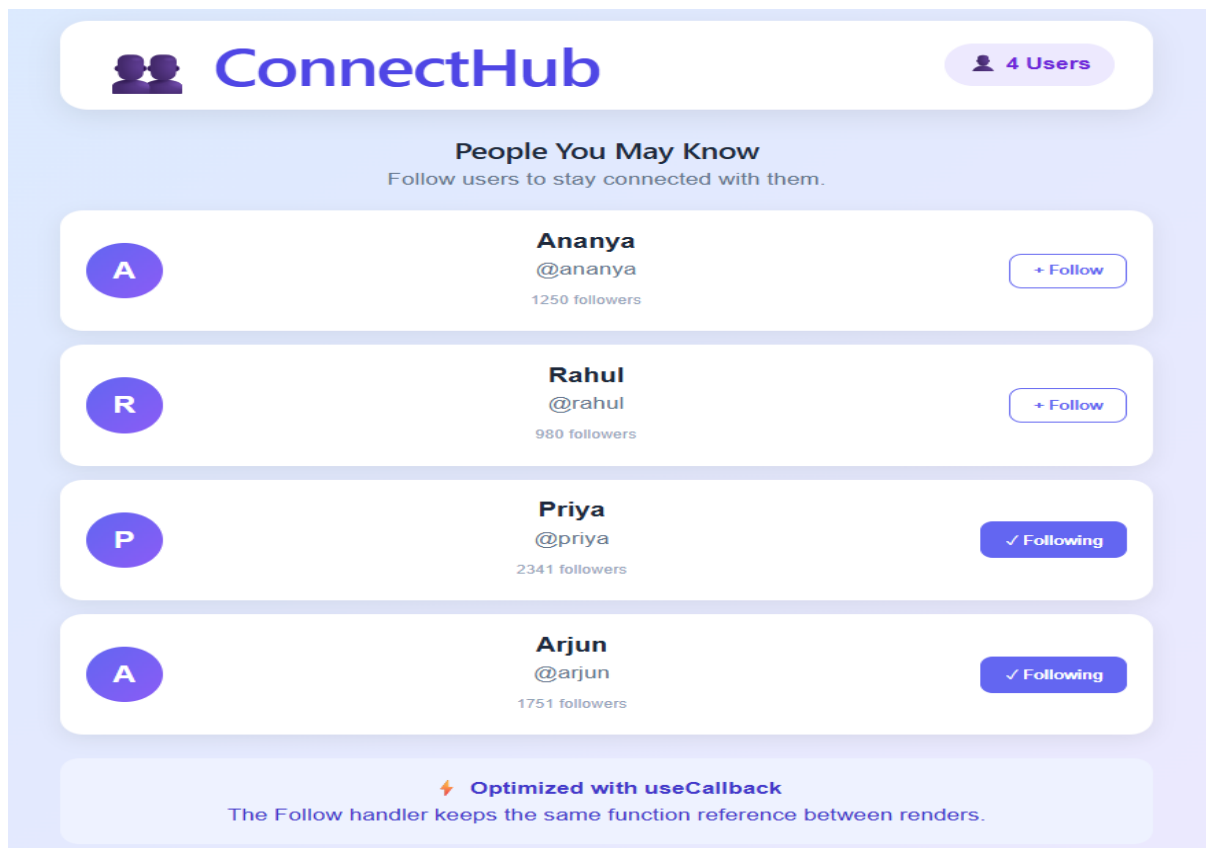
```
);
```

```
}
```

```
export default App;
```

OUTPUT:





2. You are working on a form with multiple fields. Each input has an onChange handler passed from the parent. How can useCallback help reduce unnecessary re-renders of these input components?

PROGRAM:

App.jsx:

```
import React, { useState, useCallback, memo } from "react";
import "./App.css";

const FormInput = memo(function FormInput({
  label,
  name,
  value,
  onChange,
  type = "text",
  placeholder
}) {
  console.log("Rendered:", name);
```

```
return (  
  <div className="input-group">  
    <label>{label}</label>  
    <input  
      type={type}  
      name={name}  
      value={value}  
      onChange={onChange}  
      placeholder={placeholder}  
    />  
  </div>  
);  
});  
function App() {  
  const [formData, setFormData] = useState({  
    name: "",  
    email: "",  
    phone: "",  
    password: ""  
  });  
  const handleChange = useCallback((e) => {  
    const { name, value } = e.target;  
    setFormData((previousData) => ({  
      ...previousData,  
      [name]: value  
    }));  
  }, []);  
  const handleSubmit = (e) => {  
    e.preventDefault();
```

```
    alert("Registration Successful! 🎉");
  };
  return (
    <div className="page">
      <div className="form-card">
        <div className="icon">
          
        </div>
        <h1>Create Account</h1>
        <p className="subtitle">
          Fill in your details to get started
        </p>
        <form onSubmit={handleSubmit}>
          <FormInput
            label="Full Name"
            name="name"
            value={formData.name}
            onChange={handleChange}
            placeholder="Enter your full name"
          />
          <FormInput
            label="Email Address"
            name="email"
            value={formData.email}
            onChange={handleChange}
            type="email"
            placeholder="Enter your email"
          />
          <FormInput
```

```
    label="Phone Number"
    name="phone"
    value={formData.phone}
    onChange={handleChange}
    type="tel"
    placeholder="Enter your phone number"
  />

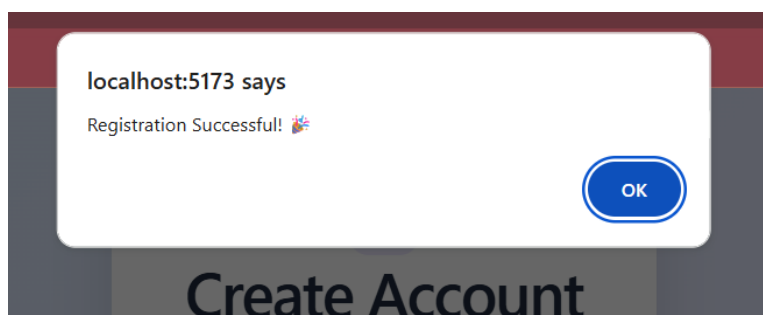
  <FormInput
    label="Password"
    name="password"
    value={formData.password}
    onChange={handleChange}
    type="password"
    placeholder="Create a password"
  />

  <button type="submit">
    Create Account →
  </button>
</form>

<div className="optimization">
  ⚡ <b>Performance Optimized</b>
  <br />
  Input components use <b>React.memo</b> and the
  change handler uses <b>useCallback</b>.
</div>
</div>
</div>
);
}
```

export default App;

OUTPUT:



3. In a to-do app, the child component renders a list of tasks and receives a deleteTask function from the parent. How can you ensure this function reference stays stable across renders using useCallback?

PROGRAM:

App.jsx:

```
import React, { useState, useCallback, memo } from "react";
import "../App.css";
const TaskList = memo(function TaskList({ tasks, deleteTask }) {
  console.log("TaskList rendered");
  return (
```

```

<div className="task-list">
  {tasks.length === 0 ? (
    <div className="empty">
      
      <h3>No tasks yet</h3>
      <p>Add a task to get started!</p>
    </div>
  ) : (
    tasks.map((task) => (
      <div className="task" key={task.id}>
        <div className="task-left">
          <div className="check">✓</div>
          <span>{task.text}</span>
        </div>
        <button
          className="delete-btn"
          onClick={() => deleteTask(task.id)}
        >
          
        </button>
      </div>
    ))
  )}
</div>

);
});

function App() {
  const [tasks, setTasks] = useState([
    { id: 1, text: "Complete React assignment" },

```

```
{ id: 2, text: "Practice useCallback" },
{ id: 3, text: "Prepare for practical exam" }
]);
const [newTask, setNewTask] = useState("");
const deleteTask = useCallback((id) => {
  setTasks((previousTasks) =>
    previousTasks.filter((task) => task.id !== id)
  );
}, []);
const addTask = () => {
  if (newTask.trim() === "") {
    return;
  }
  setTasks((previousTasks) => [
    ...previousTasks,
    {
      id: Date.now(),
      text: newTask
    }
  ]);
  setNewTask("");
};
return (
  <div className="page">
    <div className="todo-container">
      { /* Header */ }
      <div className="header">
        <div>
          <div className="logo">✓</div>
```

```
</div>

<div>

  <h1>My Tasks</h1>

  <p>Stay organized and productive</p>

</div>

<div className="task-count">

  {tasks.length}

</div>

</div>

{/* Add Task */}

<div className="add-box">

  <input

    type="text"

    value={newTask}

    placeholder="What needs to be done?"

    onChange={(e) => setNewTask(e.target.value)}

    onKeyDown={(e) => {

      if (e.key === "Enter") {

        addTask();

      }

    }}

  />

  <button className="add-btn" onClick={addTask}>

    + Add Task

  </button>

</div>

{/* Child Component */}

<TaskList

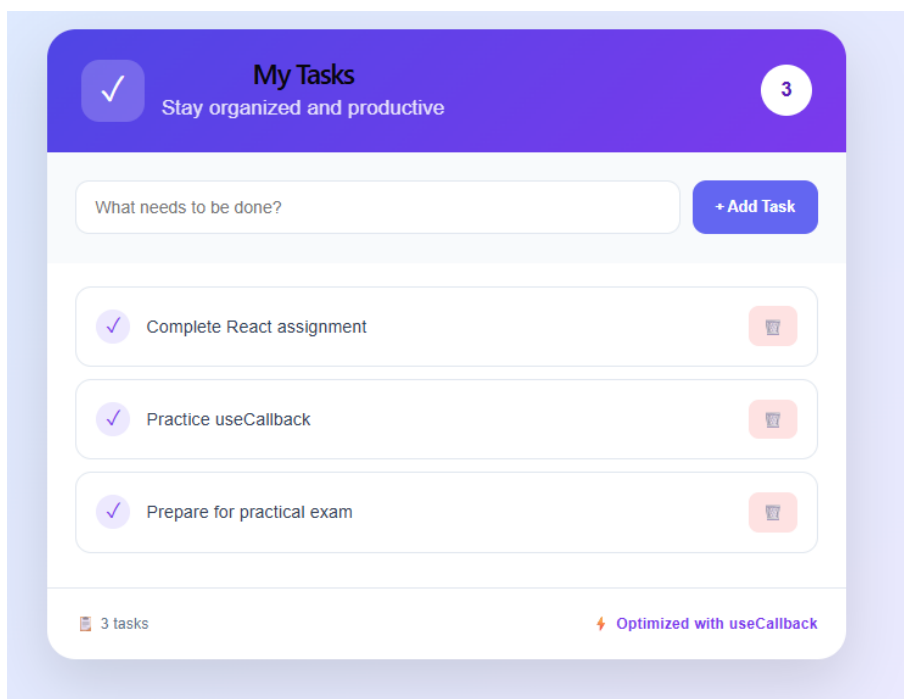
  tasks={tasks}
```

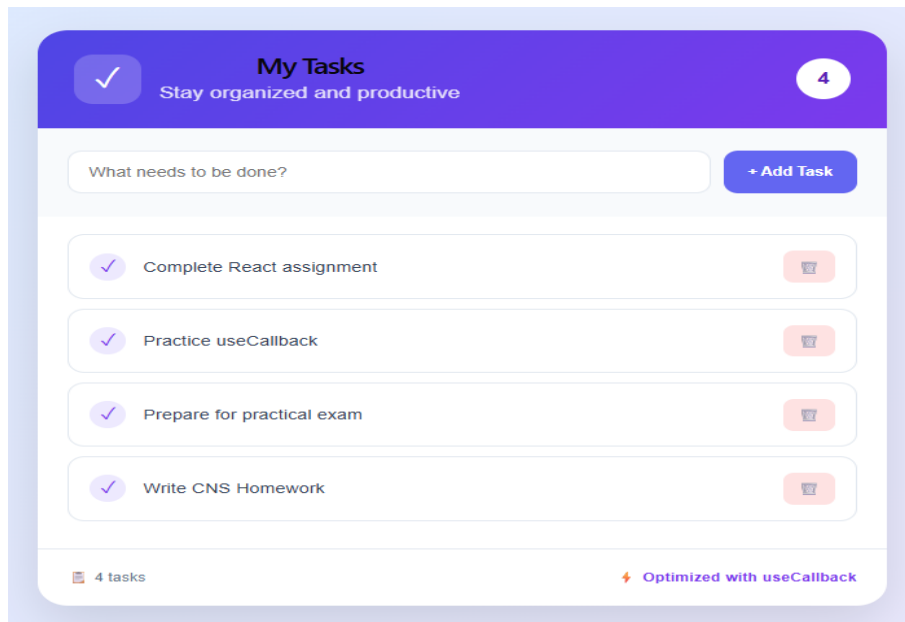
```

      deleteTask={deleteTask}
    />
    { /* Footer */ }
    <div className="footer">
      <span>
        📋 {tasks.length} {tasks.length === 1 ? "task" : "tasks"}
      </span>
      <span>
        ⚡ Optimized with useCallback
      </span>
    </div>
  </div>
</div>
);
}

export default App;

```

OUTPUT:



E) useMemo

1. In a financial dashboard, calculating the total revenue requires iterating over thousands of records. How can you prevent recalculating it on every render using useMemo?

PROGRAM:

App.jsx:

```
import React, { useState, useMemo } from "react";
import "./App.css";

function App() {
  const [transactions] = useState([
    { id: 1, customer: "John", amount: 25000 },
    { id: 2, customer: "Sarah", amount: 18500 },
    { id: 3, customer: "David", amount: 32000 },
    { id: 4, customer: "Emma", amount: 27500 },
    { id: 5, customer: "Michael", amount: 41000 }
  ]);

  const [count, setCount] = useState(0);

  const totalRevenue = useMemo(() => {
    console.log("Calculating total revenue...");
    return transactions.reduce(
```

```
(total, transaction) =>
  total + transaction.amount,
  0
);
}, [transactions]);

const averageRevenue = useMemo(() => {
  return totalRevenue / transactions.length;
}, [totalRevenue, transactions]);

return (
  <div className="page">
    <div className="dashboard">
      {/* HEADER */}
      <header>
        <div>
          <h1>💰 FinancePro</h1>
          <p>Financial Revenue Dashboard</p>
        </div>
        <div className="status">
          ● LIVE
        </div>
      </header>
      {/* SUMMARY */}
      <div className="summary">
        <div className="card revenue">
          <div className="card-icon">
            💵
          </div>
          <div>
            <p>Total Revenue</p>
```

```
<h2>
    ₹ {totalRevenue.toLocaleString()}
</h2>
<span>↗ 12.5% this month</span>
</div>
</div>
<div className="card">
    <div className="card-icon">
        
    </div>
    <div>
        <p>Transactions</p>
        <h2>
            {transactions.length}
        </h2>
        <span>Successful payments</span>
    </div>
</div>
<div className="card">
    <div className="card-icon">
        
    </div>
    <div>
        <p>Average Revenue</p>
        <h2>
            ₹ {Math.round(averageRevenue).toLocaleString()}
        </h2>
        <span>Per transaction</span>
    </div>
</div>
```

```
</div>

</div>

{ /* TRANSACTIONS */ }

<section className="transactions">

  <div className="section-title">

    <div>

      <h2>Recent Transactions</h2>

      <p>Revenue generated from customers</p>

    </div>

    <div className="transaction-count">

      {transactions.length} Records

    </div>

  </div>

  <div className="table">

    <div className="table-header">

      <span>ID</span>

      <span>Customer</span>

      <span>Amount</span>

      <span>Status</span>

    </div>

    {transactions.map((transaction) => (

      <div className="table-row" key={transaction.id}>

        <span className="id">

          #{transaction.id}

        </span>

        <span className="customer">

           {transaction.customer}

        </span>

        <span className="amount">
```

```
    ₹{transaction.amount.toLocaleString()}
  </span>
  <span className="paid">
    ✓ Paid
  </span>
</div>

  )})
</div>
</section>
{ /* TEST BUTTON */}
<div className="test-box">
  <div>
    <strong> ⚡ Test useMemo</strong>
    <p>
      Clicking this button re-renders the component,
      but the revenue calculation is not repeated.
    </p>
  </div>
  <button onClick={() => setCount(count + 1)}>
    Re-render ({count})
  </button>
</div>
{ /* FOOTER */}
<footer>
  <span>© 2026 FinancePro</span>
  <span>
    ⚡ Total revenue optimized with useMemo
  </span>
</footer>
```

```

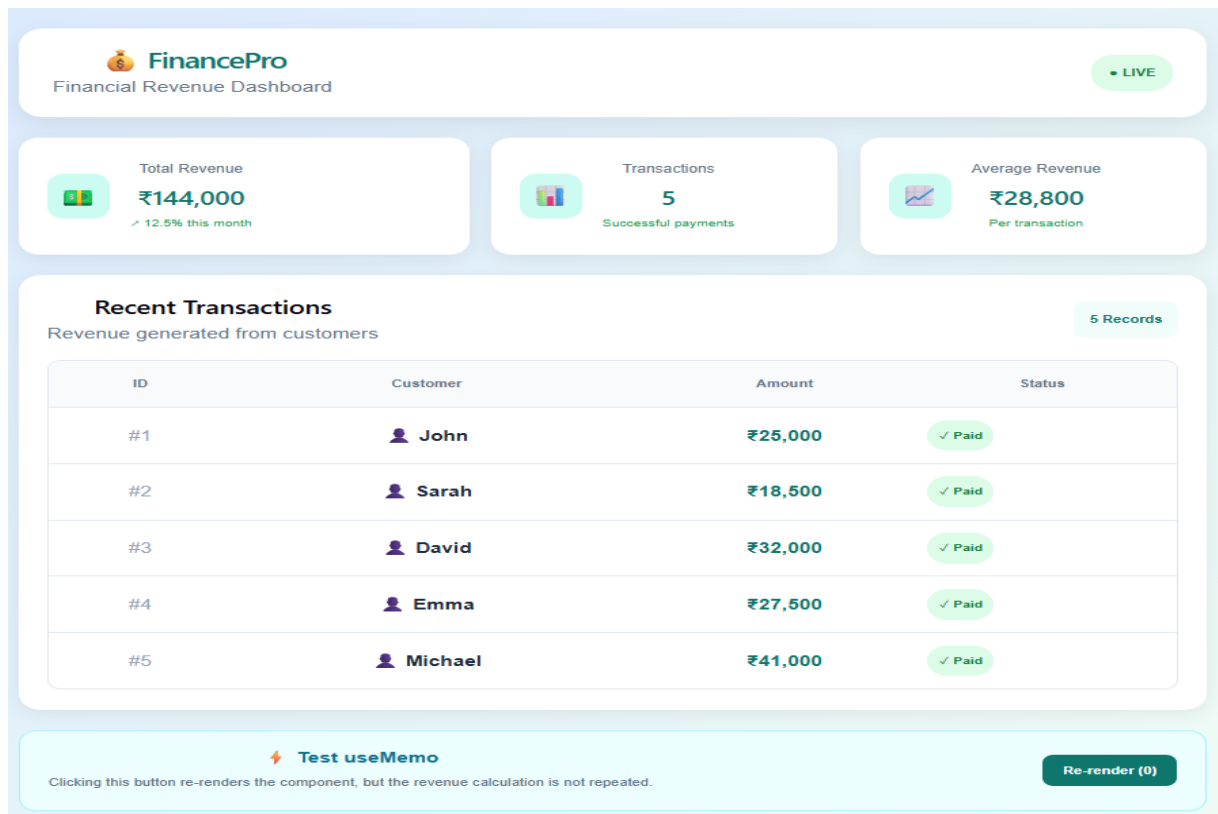
    </div>

    </div>

  );
}

export default App;

```

OUTPUT:

2. You are building a search bar that filters a large dataset of products. The filtering logic is slow. How can you optimize this so filtering only happens when the search term changes?

PROGRAM:**App.jsx:**

```

import React, { useState, useMemo } from "react";
import "./App.css";

function App() {
  const [searchTerm, setSearchTerm] = useState("");
  const [count, setCount] = useState(0);
  const products = [

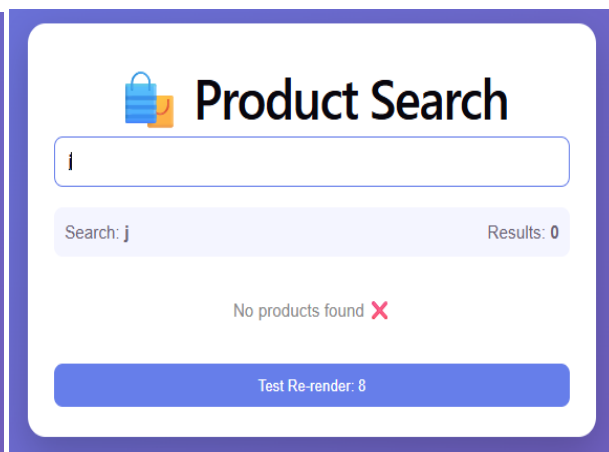
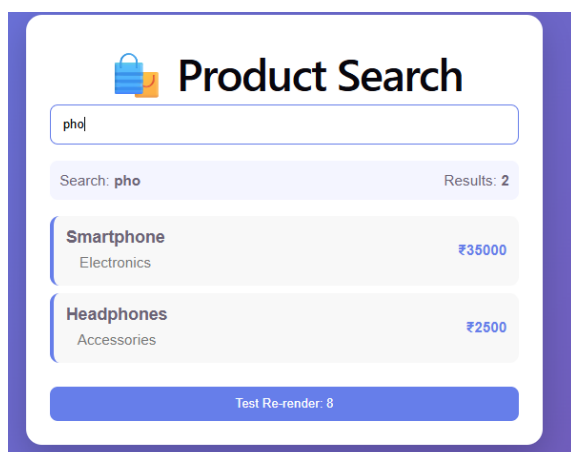
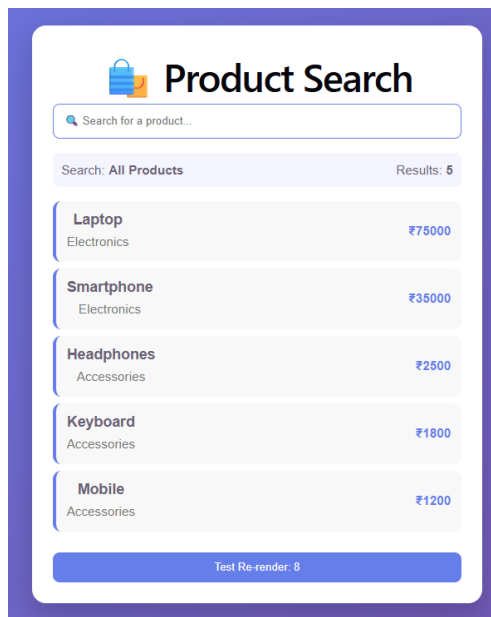
```

```
{ id: 1, name: "Laptop", category: "Electronics", price: 75000 },
{ id: 2, name: "Smartphone", category: "Electronics", price: 35000 },
{ id: 3, name: "Headphones", category: "Accessories", price: 2500 },
{ id: 4, name: "Keyboard", category: "Accessories", price: 1800 },
{ id: 5, name: "Mobile", category: "Accessories", price: 1200 },
];

const filteredProducts = useMemo(() => {
  console.log("Filtering products...");
  return products.filter((product) =>
    product.name.toLowerCase().includes(searchTerm.toLowerCase())
  );
}, [searchTerm]);

return (
  <div className="container">
    <div className="card">
      <h1>🛒 Product Search</h1>
      <p className="subtitle">
        </p>
      <input
        type="text"
        placeholder="🔍 Search for a product..."
        value={searchTerm}
        onChange={(e) => setSearchTerm(e.target.value)}
      />
      <div className="info">
        <span>
          Search: <b>{searchTerm || "All Products"}</b>
        </span>
        <span>
```

```
Results: <b>{filteredProducts.length}</b>
</span>
</div>
<div className="products">
  {filteredProducts.length > 0 ? (
    filteredProducts.map((product) => (
      <div className="product" key={product.id}>
        <div>
          <h3>{product.name}</h3>
          <p>{product.category}</p>
        </div>
        <strong>₹{product.price}</strong>
      </div>
    ))
  ) : (
    <p className="no-result">No products found ❌</p>
  )}
</div>
<button onClick={() => setCount(count + 1)}>
  Test Re-render: {count}
</button>
</div>
</div>
);
}
export default App;
```

OUTPUT:**F) useRef**

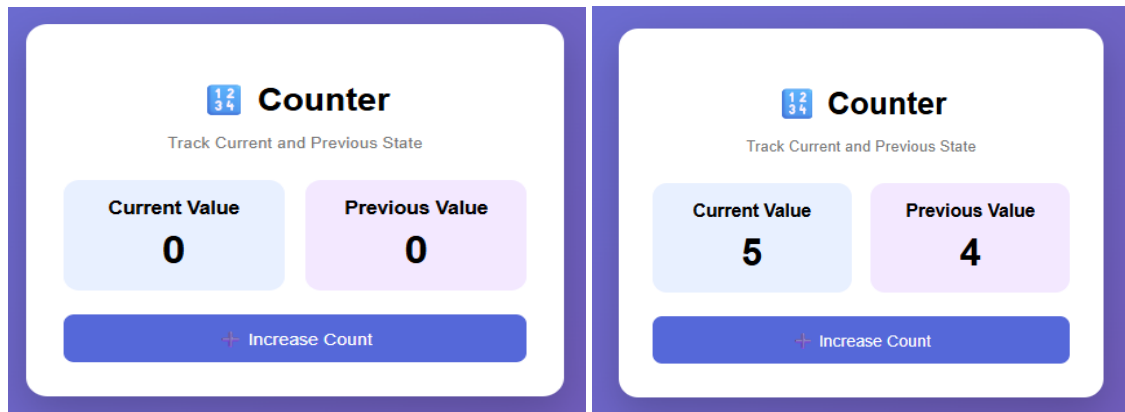
1. Track Previous State : Create a counter (count) that increases on button click. Use useRef to store the previous value of count. Display both the current and previous values.

PROGRAM:**App.jsx:**

```
import React, { useState, useRef, useEffect } from "react";
import "./App.css";
function App() {
  const [count, setCount] = useState(0);
  // Stores the previous count
  const previousCount = useRef(0);
  useEffect(() => {
    previousCount.current = count;
  }, [count]);
  return (
    <div className="container">
      <div className="card">
        <h1> 🎮 Counter</h1>
        <p className="subtitle">Track Current and Previous State</p>
        <div className="values">
          <div className="box current">
            <h3>Current Value</h3>
            <p>{count}</p>
          </div>
          <div className="box previous">
            <h3>Previous Value</h3>
            <p>{previousCount.current}</p>
          </div>
        </div>
        <button onClick={() => setCount(count + 1)}>
          + Increase Count
        </button>
      </div>
    </div>
  );
}
```

```
);
}

export default App;
```

OUTPUT:

2. Scroll to an Element: Create a long list of items. Use useRef to point to a specific item (e.g., the 50th item). Add a button that scrolls directly to that item.

PROGRAM:**App.jsx:**

```
import React, { useRef } from "react";
import "./App.css";

function App() {
  const item50Ref = useRef(null);
  const scrollToItem50 = () => {
    item50Ref.current.scrollIntoView({
      behavior: "smooth",
      block: "center"
    });
  };

  return (
    <div className="container">
```

```
<h1> 📄 Long Item List</h1>

<button onClick={scrollToItem50}>

  🎯 Go to Item 50

</button>

<div className="list">

  {Array.from({ length: 100 }, (_, index) => {

    const itemNumber = index + 1;

    return (

      <div

        key={itemNumber}

        ref={itemNumber === 50 ? item50Ref : null}

        className={`item ${

          itemNumber === 50 ? "target" : ""

        }}`

      >

        Item {itemNumber}

        {itemNumber === 50 && (

          <span> 🌟 Target Item</span>

        )}

      </div>

    );

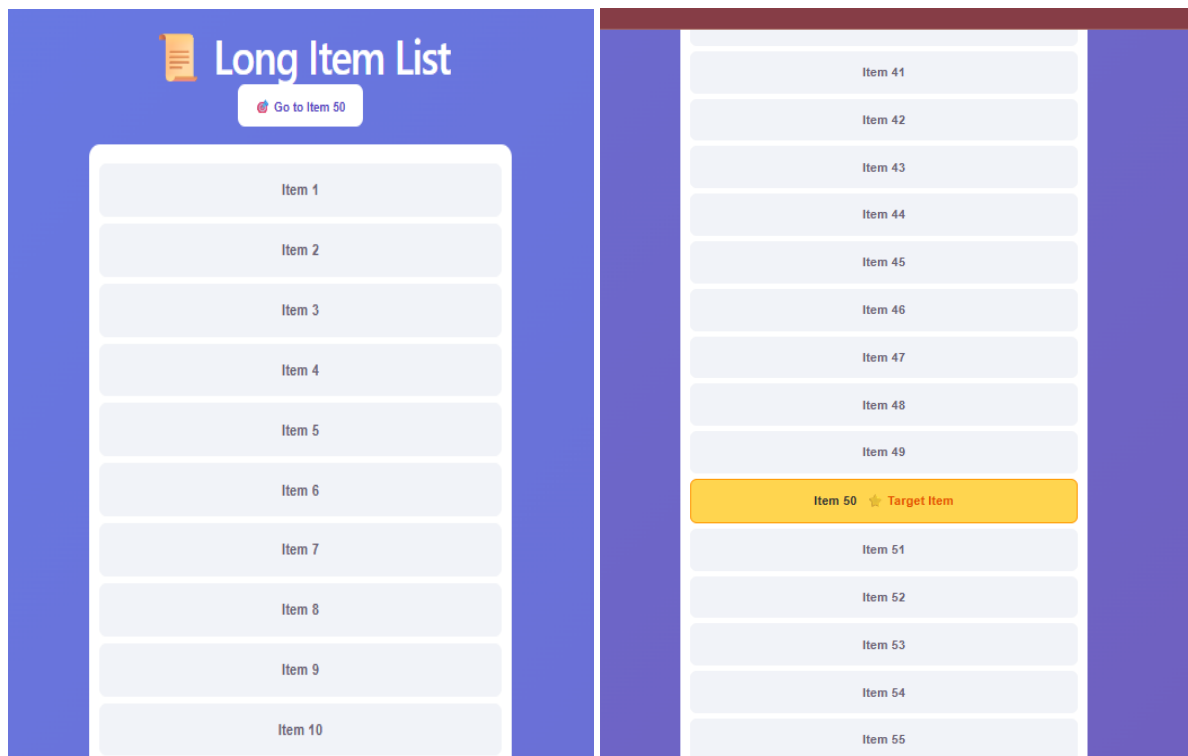
  )}

</div>

);

}

export default App;
```

OUTPUT:**G) Customized Hooks**

useDebounce Hook: Accept a value and delay. Return the debounced value (only update after no changes for given delay).

App.jsx

```
import React, { useState } from "react";
import useDebounce from "../useDebounce";
import "../App.css";

function App() {
  const [search, setSearch] = useState("");

  // Debounce for 1 second
  const debouncedSearch = useDebounce(search, 1000);

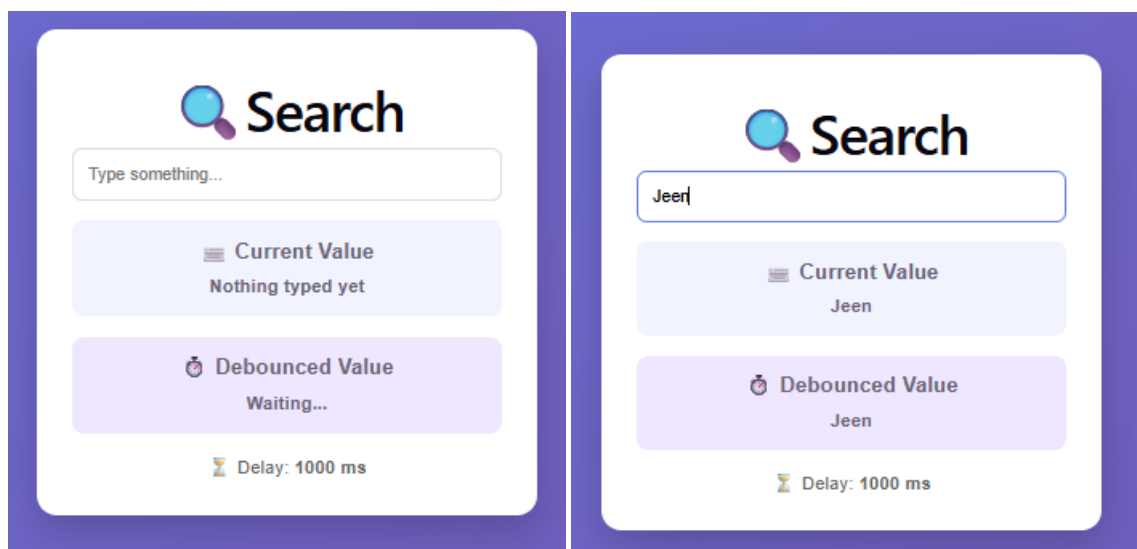
  return (
    <div className="container">
      <div className="card">
        <h1>🔍 Search</h1>
        <p className="subtitle">
```

```
</p>
<input
  type="text"
  placeholder="Type something..."
  value={search}
  onChange={(e) => setSearch(e.target.value)}
/>
<div className="result">
  <h3>📝 Current Value</h3>
  <p>{search || "Nothing typed yet"}</p>
</div>
<div className="result debounced">
  <h3>🕒 Debounced Value</h3>
  <p>{debouncedSearch || "Waiting..."}</p>
</div>
<div className="delay">
  ⌚ Delay: <b>1000 ms</b>
</div>
</div>
</div>
);
}
export default App;
useDebounce.jsx
import { useState, useEffect } from "react";
function useDebounce(value, delay) {
  const [debouncedValue, setDebouncedValue] = useState(value);
  useEffect(() => {
    const timer = setTimeout(() => {
```

```

    setDebounceValue(value);
  }, delay);
  return () => {
    clearTimeout(timer);
  };
}, [value, delay]);
return debounceValue;
}
export default useDebounce;

```

OUTPUT:

2. useEventListener Hook: Create a hook that attaches/detaches an event listener to any target (window, document, ref.current).

PROGRAM:**App.jsx:**

```

import React, { useState, useRef } from "react";
import useEventListener from "../useEventListener";
import "../App.css";

function App() {
  const [width, setWidth] = useState(window.innerWidth);
  const [key, setKey] = useState("None");

```

```
const [mouseMessage, setMouseMessage] = useState(
  "Move your mouse inside the box"
);
const boxRef = useRef(null);
const handleResize = () => {
  setWidth(window.innerWidth);
};
useEventListener("resize", handleResize, window);
const handleKeyDown = (event) => {
  setKey(event.key);
};
useEventListener("keydown", handleKeyDown, document);
const handleMouseEnter = () => {
  setMouseMessage(" 🖱 Mouse is inside the box!");
};
const handleMouseLeave = () => {
  setMouseMessage("Move your mouse inside the box");
};
useEventListener("mouseenter", handleMouseEnter, boxRef.current);
useEventListener("mouseleave", handleMouseLeave, boxRef.current);
return (
  <div className="container">
    <div className="card">
      <h1> ⚡ useEventListener</h1>
      <p className="subtitle">
        </p>
      <div className="section">
        <h2> 🖱 Window Event</h2>
        <p>
```

```

    Current Window Width:
    <strong> {width}px</strong>
  </p>
  <small>Resize your browser window</small>
</div>

<div className="section">
  <h2> 🖱️ Document Event</h2>
  <p>
    Last Key Pressed:
    <strong> {key}</strong>
  </p>
  <small>Press any keyboard key</small>
</div>

<div
  ref={boxRef}
  className="mouse-box"
>
  <h2> 🖱️ Ref Element Event</h2>
  <p>{mouseMessage}</p>
</div>
</div>
</div>
);
}

export default App;

useEventListener.js

import { useEffect } from "react";

function useEventListener(eventName, handler, target = window) {
  useEffect(() => {

```

```

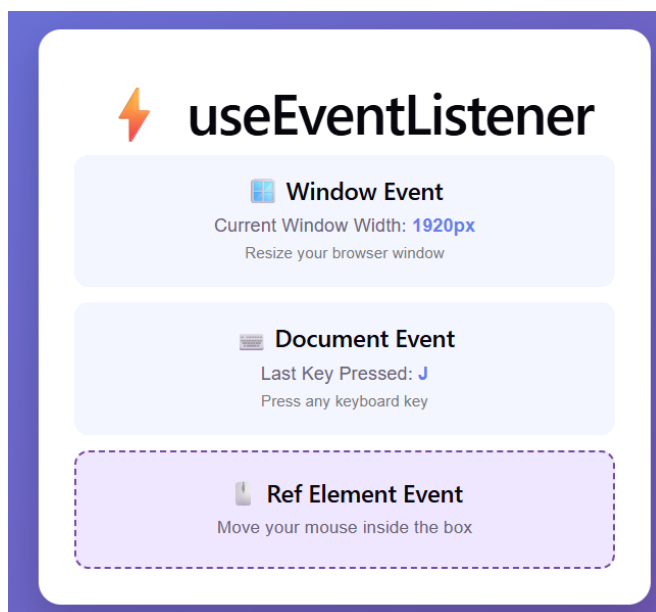
if (!target) return;

target.addEventListener(eventName, handler);

return () => {
  target.removeEventListener(eventName, handler);
};
}, [eventName, handler, target]);
}

export default useEventListener;

```

OUTPUT:

Problem Understanding & Approach (10)	Implementation & Execution (15)	Tool/Technology Usage (10)	Code Quality & Documentation (5)	Viva (5)	Time Management (5)	Total (50)

Result:

Thus, the program using useState, useEffect, useContext, useCallback, useMemo, useRef and Customized Hooks is completed and the output is verified.